

Suveniriri

Amaru kupuje suvenire u stranoj prodavnici. Postoji N **vrsta** suvenira. U prodavnici je dostupan beskonačan broj suvenira svake vrste.

Svaka vrsta suvenira ima fiksnu cijenu. Naime, suvenir tipa i ($0 \leq i < N$) ima cijenu od $P[i]$ kovanica, gdje je $P[i]$ pozitivan cijeli broj.

Amaru zna da su vrste suvenira sortirane po opadajućem redoslijedu cijene, i da su cijene suvenira različite. Konkretno, $P[0] > P[1] > \dots > P[N-1] > 0$. Štaviše, uspio je saznati vrijednost $P[0]$. Nažalost, Amaru nema nikakve druge informacije o cijenama suvenira.

Da bi kupio suvenire, Amaru će obaviti niz transakcija s prodavačem.

Svaka transakcija se sastoji od sljedećih koraka:

1. Amaru predaje određeni (pozitivan) broj novčića prodavcu.
2. Prodavac stavlja ove novčiće na hrpu na stolu u zadnjoj sobi, gdje ih Amaru ne može vidjeti.
3. Prodavac razmatra svaki tip suvenira $0, 1, \dots, N-1$ tim redom, jedan po jedan. Svaki tip se smatra tačno jednom po transakciji.
 - Prilikom razmatranja suvenira tipa i , Ako je trenutni broj novčića u hrpi najmanje $P[i]$, onda
 - prodavač uklanja $P[i]$ novčića iz hrpe, i
 - prodavač stavlja jedan suvenir tipa i na stol.
4. Prodavac uzima sve preostale kovanice u hrpi i sve suvenire sa stola i daje ih Amaru.

Imajte na umu da na stolu nema kovanica ili suvenira prije početka transakcije.

Vaš zadatak je da uputite Amaru da izvrši određeni broj transakcija, tako da:

- u svakoj transakciji kupi **barem jedan** suvenir, i
- ukupno kupi **tačno** i suvenira tipa i , za svako i takvo da $0 \leq i < N$. Imajte na umu da ovo znači da Amaru ne bi trebao kupovati suvenire tipa 0.

Amaru ne mora minimizirati broj transakcija i ima neograničen broj zaliha novčića.

Detalji implementacije

Trebali biste implementirati sljedeći postupak.

```
void buy_souvenirs(int N, long long P0)
```

- N : broj vrsta suvenira.
- $P0$: vrijednost $P[0]$.
- Ova procedura se poziva tačno jednom za svaki testni slučaj.

Gornja procedura može pozivati sljedeću proceduru da uputi Amaru da izvrši transakciju:

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- M : broj novčića koje je Amaru predao prodavcu.
- Procedura vraća par. Prvi element para je niz L , koji sadrže vrste suvenira koji su kupljeni (u rastućem redoslijedu). Drugi element je cijeli broj R , broj novčića vraćenih Amaru nakon transakcije.
- Potrebno je da $P[0] > M \geq P[N - 1]$. Uslov $P[0] > M$ osigurava da Amaru ne kupuje nijedan suvenir tipa 0, a $M \geq P[N - 1]$ osigurava da Amaru kupi barem jedan suvenir. Ako ovi uslovi nisu ispunjeni, vaše rješenje će dobiti presudu `Output isn't correct: Invalid argument`. Imajte na umu da, suprotno od $P[0]$, vrijednost $P[N - 1]$ nije navedena u ulazu (inputu).
- Procedura se može pozvati najviše 5000 puta u svakom testnom slučaju.

Ponašanje grejderskog uređaja **nije adaptivno**. To znači da je sekvenca, niz cijena P fiksna prije nego što se pozove funkcija `buy_souvenirs`.

Ograničenja

- $2 \leq N \leq 100$
- $1 \leq P[i] \leq 10^{15}$ za svako i tako da $0 \leq i < N$.
- $P[i] > P[i + 1]$ za svako i tako da $0 \leq i < N - 1$.

Podzadaci

Podzadatak	Rezultat	Dodatna ograničenja
1	4	$N = 2$
2	3	$P[i] = N - i$ za svako i takvo da $0 \leq i < N$.
3	14	$P[i] \leq P[i + 1] + 2$ za svako i takvo da je $0 \leq i < N - 1$.
4	18	$N = 3$
5	28	$P[i + 1] + P[i + 2] \leq P[i]$ za svako i tako da je $0 \leq i < N - 2$. $P[i] \leq 2 \cdot P[i + 1]$ za svako i tako da je $0 \leq i < N - 1$.
6	33	Nema dodatnih ograničenja.

Primjer

Razmotrite sljedeći poziv.

```
buy_souvenirs(3, 4)
```

Postoji $N = 3$ vrsta suvenira i $P[0] = 4$. Primijetite da postoje samo tri moguća niza cijena P : $[4, 3, 2]$, $[4, 3, 1]$, i $[4, 2, 1]$.

Pretpostavimo da `buy_souvenirs` poziva `transaction(2)`. Pretpostavimo da poziv vraća $([2], 1)$, što znači da je Amaru kupio jedan suvenir od 2 i prodavač mu je vratio novčić 1. Primijetite da nam ovo omogućava da zaključimo da $P = [4, 3, 1]$, budući da:

- Za $P = [4, 3, 2]$, `transaction(2)` bi vratio $([2], 0)$.
- Za $P = [4, 2, 1]$, `transaction(2)` bi vratio $([1], 0)$.

Tada `buy_souvenirs` može pozvati `transaction(3)`, što vraća $([1], 0)$, što znači da je Amaru kupio jedan suvenir od 1 i prodavač mu je vratio kovanice 0. Do sada, ukupno, Kupio je jedan suvenir tipa 1 i jedan suvenir tipa 2.

Konačno, `buy_souvenirs` može pozvati `transaction(1)`, što vraća $([2], 0)$, što znači da je Amaru kupio jedan suvenir vrste 2. Imajte na umu da smo ovdje mogli koristiti i `transaction(2)`. U ovom trenutku, ukupno Amaru ima jedan suvenir tipa 1 i dva suvenira tipa 2, po potrebi.

Primjer ocjenjivača

Format unosa:

```
N
P[0] P[1] ... P[N-1]
```

Izlazni format:

```
Q[0] Q[1] ... Q[N-1]
```

Ovdje je $Q[i]$ ukupan broj kupljenih suvenira tipa i za svako i tako da $0 \leq i < N$.